Report on Homework 1

——Machine Learning

刘滨瑞

2021012579　未央-水木 12

2025 年 3 月 13 日

2　线性模型与梯度下降

2.1　特征归一化

已在代码中实现，参见./SGD/startcode.py 中的 feature_normalization 函数。

2.2　目标函数与梯度

2.2.1　目标函数的矩阵形式

$$J(\boldsymbol{\theta}) = \frac{1}{m}\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 + \lambda\boldsymbol{\theta}^T\boldsymbol{\theta} \tag{1}$$

2.2.2　实现函数 compute_regularized_square_loss

函数已实现，参见./SGD/startcode.py 文件中的相应函数。

2.2.3 目标函数的梯度

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{m}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta} \tag{2}$$

2.2.4 实现函数 compute_regularized_square_loss_gradient

函数已实现，参见./SGD/startcode.py 文件中的相应函数。

2.2.5 梯度检查

已在代码中实现，参见./SGD/startcode.py 文件中的 grad_checker 函数。

2.2.6 降低正则化对偏置项的影响

设 $\boldsymbol{X} = \begin{bmatrix}\boldsymbol{X}' & \boldsymbol{B}\end{bmatrix}$，其中 $\boldsymbol{X}'$ 是未添加偏置项的数据矩阵，$\boldsymbol{B} \in \mathbb{R}^m$ 代表偏置项，为所有值均等于 B 的向量。根据定义，有 $\boldsymbol{X}'\boldsymbol{\theta}' + \begin{bmatrix} b \\ ... \\ b \end{bmatrix} = \hat{\boldsymbol{y}}$ 成立，其中 $\hat{\boldsymbol{y}}$ 是模型的预测值。

设 $\boldsymbol{\theta} = \begin{bmatrix}\boldsymbol{\theta}' \\ \theta_B\end{bmatrix}$，其中 $\boldsymbol{\theta}'$ 是未添加偏置项的权重向量，θ_B 代表偏置项，且满足代数关系：$\theta_B B = b$。

代入目标函数的梯度 (2) 式中，有：

$$\begin{aligned}\nabla J(\boldsymbol{\theta}) &= \frac{2}{m}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta} \\ &= \frac{2}{m}\begin{bmatrix}\boldsymbol{X}'^T \\ \boldsymbol{B}^T\end{bmatrix}(\begin{bmatrix}\boldsymbol{X}' & \boldsymbol{B}\end{bmatrix}\begin{bmatrix}\boldsymbol{\theta}' \\ \theta_B\end{bmatrix} - \boldsymbol{y}) + 2\lambda\begin{bmatrix}\boldsymbol{\theta}' \\ \theta_B\end{bmatrix}\end{aligned}$$

我们关注目标函数梯度中的偏置项，即 $[\nabla J(\boldsymbol{\theta})]_{d+1}$ 的值，它决定了梯度下降算法中偏置项 b 的变化速度。

$$[\nabla J(\boldsymbol{\theta})]_{d+1} = \frac{2}{m}(\boldsymbol{B}^T\boldsymbol{X}'\boldsymbol{\theta}' + \boldsymbol{B}^T\boldsymbol{B}\theta_B - \boldsymbol{B}^T\boldsymbol{y}) + 2\lambda\theta_B$$

$$
\begin{aligned}
&= [\frac{2}{m}\boldsymbol{B}^T(\boldsymbol{X}'\boldsymbol{\theta}' + \begin{bmatrix} b \\ ... \\ b \end{bmatrix} - \boldsymbol{y})] + [2\lambda\frac{b}{B}] \\
&= [2B\text{mean}(\hat{\boldsymbol{y}} - \boldsymbol{y})] + [2\lambda\frac{b}{B}]
\end{aligned}
$$

其中 $2\lambda\theta_B$ 代表了正则化对偏置项的影响，我们设其占总体 $[\nabla J(\boldsymbol{\theta})]_{d+1}$ 的比例为 γ，则有：

$$
\begin{aligned}
\gamma &= \frac{2\lambda\frac{b}{B}}{[2B\text{mean}(\hat{\boldsymbol{y}} - \boldsymbol{y})] + [2\lambda\frac{b}{B}]} \\
&= \frac{1}{1 + B^2\frac{\text{mean}(\hat{\boldsymbol{y}} - \boldsymbol{y})}{b\lambda}}
\end{aligned}
$$

显然，随着 B 的增大，γ 的值会迅速减小至 0，这即证明了将额外维度 B 设置为一个更大的数确实可以降低正则化对偏置项的影响。

2.3 梯度下降

2.3.1 梯度下降中目标函数值的变化

我们在 $\boldsymbol{\theta}$ 附近作优化目标函数 J 的一阶 Taylor 展开。

$$
J(\boldsymbol{\theta}') = J(\boldsymbol{\theta}) + (\boldsymbol{\theta}' - \boldsymbol{\theta})^T\nabla J(\boldsymbol{\theta})
$$

取 $\boldsymbol{\theta}' = \boldsymbol{\theta} + \eta\boldsymbol{h}$，有：

$$
J(\boldsymbol{\theta} + \eta\boldsymbol{h}) - J(\boldsymbol{\theta}) = \eta\boldsymbol{h}^T\nabla J(\boldsymbol{\theta}) \tag{3}
$$

我们希望取适当方向上的 $\boldsymbol{h}$，使得目标函数的下降速度最快。

由于 $\eta > 0$，$\boldsymbol{h}$ 应满足：

$$
\boldsymbol{h} = \arg\min_{\boldsymbol{h}}(\frac{1}{\|\boldsymbol{h}\|_2}\boldsymbol{h}^T\nabla J(\boldsymbol{\theta}))
$$

若设 $\boldsymbol{h}$ 和 $\nabla J(\boldsymbol{\theta})$ 在欧氏空间中的方向角为 α，我们有不等关系：

$$\frac{1}{\|\boldsymbol{h}\|_2}\boldsymbol{h}^T\nabla J(\boldsymbol{\theta}) = \frac{1}{\|\boldsymbol{h}\|_2}\|\boldsymbol{h}\|_2\|\nabla J(\boldsymbol{\theta})\|_2\cos\alpha \geq -\|\nabla J(\boldsymbol{\theta})\|_2$$

其中等号当且仅当 $\alpha = \pi$ 时取到，此时 $\boldsymbol{h}$ 恰好与 $\nabla J(\boldsymbol{\theta})$ 反向。

综上，当 $\boldsymbol{h}$ 与 $-\nabla J(\boldsymbol{\theta})$ 平行时，目标函数的下降速度最快。

2.3.2 权重更新的表达式

在梯度下降过程中，记第 t 次迭代后的 $\boldsymbol{\theta}$ 为 $\boldsymbol{\theta}^t$，则权重更新的公式为

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \eta\nabla J(\boldsymbol{\theta}^t) \tag{4}$$

在本问题中，将梯度表示代入，可得：

$$\boldsymbol{\theta}^{t+1} = (1 - 2\eta\lambda - \frac{2\eta}{m}\boldsymbol{X}^T\boldsymbol{X})\boldsymbol{\theta}^t + \frac{2\eta}{m}\boldsymbol{X}^T\boldsymbol{y}$$

2.3.3 梯度下降算法的实现

参见./SGD/startcode.py 文件中的 batch_grad_descent 函数。

2.3.4 步长参数调整

固定 $\lambda = 0$，调整 η 的值，观察梯度下降算法的收敛情况。结果如图 1、图 2 所示。

我们测试了步长 η 为 0.001、0.002、0.005、0.010、0.050、0.100、0.500 和 1.000 的情况。从图中可以看出，当 $\eta \geq 0.100$ 时，梯度下降算法会发散，且 η 越大，发散就越快发生。而当 $\eta \leq 0.050$ 时，梯度下降算法能够收敛。在该范围内，η 越大，梯度下降算法的收敛速度越快。

在我们测试的步长中，$\eta = 0.050$ 时的收敛速度是最快的，经过约 2000 个 epoch 就已经收敛了。

我们估计，在 $(0.050, 0.100)$ 间存在着某个更优的 η 值，使得模型的收敛速度更快。

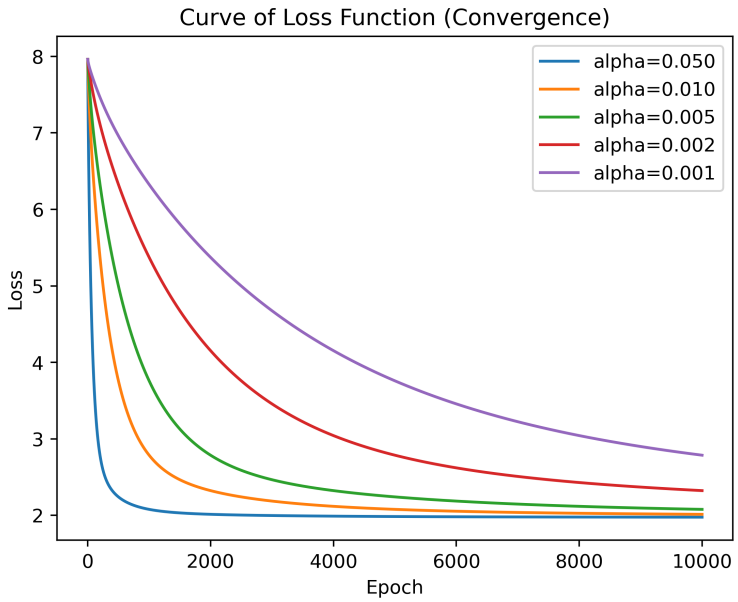

图 1: 不同步长下训练集上的损失函数的变化曲线

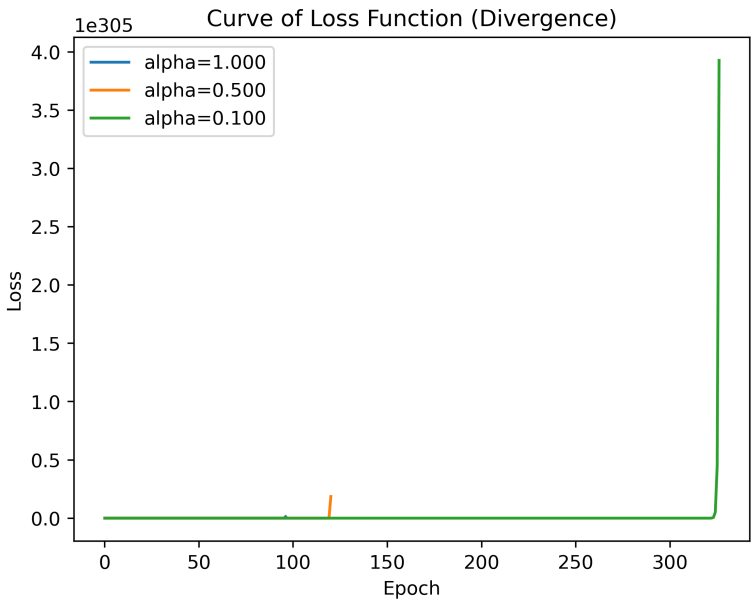

图 2: 不同步长下训练集上的损失函数的变化曲线

2.4 随机梯度下降

2.4.1 SGD 算法中梯度的表达式

对于自 $\{1,2,...,m\}$ 中等概率地独立同分布采样得到的 $q_1,q_2,...,q_n$，我们定义抽样矩阵 $\boldsymbol{S}=(s_{ij})\in\mathbb{R}^{n\times m}$, 满足 $\forall i\in\{1,2,...,n\},\forall j\in\{1,2,...,m\}$, 有:

$$s_{ij}=\begin{cases}0 & \text{if} \quad j\neq q_i\\ 1 & \text{if} \quad j=q_i\end{cases}$$

则随机梯度下降算法中的优化目标函数 $J_{SGD}(\boldsymbol{\theta})$ 可表示为:

$$J_{SGD}(\boldsymbol{\theta})=\frac{1}{n}\|\boldsymbol{S}\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{S}\boldsymbol{y}\|_2^2+\lambda\boldsymbol{\theta}^T\boldsymbol{\theta} \tag{5}$$

因此，梯度 $\nabla J_{SGD}(\boldsymbol{\theta})$ 的表达式为:

$$\nabla J_{SGD}(\boldsymbol{\theta})=\frac{2}{n}\boldsymbol{X}^T\boldsymbol{S}^T\boldsymbol{S}(\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y})+2\lambda\boldsymbol{\theta} \tag{6}$$

2.4.2 证明 SGD 算法下的梯度是对原梯度的无偏估计

命题 $\mathbb{E}[\nabla J_{SGD}(\boldsymbol{\theta})]=\nabla J(\boldsymbol{\theta})$，等价于:

$$\mathbb{E}[\frac{2}{n}\boldsymbol{X}^T\boldsymbol{S}^T\boldsymbol{S}(\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y})+2\lambda\boldsymbol{\theta}]=\frac{2}{m}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y})+2\lambda\boldsymbol{\theta}$$

由于期望的线性性质，我们有:

$$\mathbb{E}[\frac{2}{n}\boldsymbol{X}^T\boldsymbol{S}^T\boldsymbol{S}(\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y})+2\lambda\boldsymbol{\theta}]=\frac{2}{n}\boldsymbol{X}^T\mathbb{E}[\boldsymbol{S}^T\boldsymbol{S}](\boldsymbol{X}\boldsymbol{\theta}-\boldsymbol{y})+2\lambda\boldsymbol{\theta}$$

对比两式，我们只需证明:

$$\mathbb{E}[\boldsymbol{S}^T\boldsymbol{S}]=\frac{n}{m}\boldsymbol{I}_m$$

$\boldsymbol{S}^T\boldsymbol{S}\in\mathbb{R}^{m\times m}$。对 $\forall k,p\in\{1,2,...,m\}$，设 $\boldsymbol{S}^T\boldsymbol{S}$ 的第 k 行第 p 列元素为 r_{kp}，计算易得:

$$r_{kp}=\sum_{u=1}^{n}s_{uk}s_{up}$$

考虑 $\boldsymbol{S}$ 的定义，当 j 固定时，$s_{uj}=1$ 当且仅当 $j=q_u$，否则等于 0。因此，若 $k\neq p$，则 k、p 至少有一个不等于 q_u，于是：

$$r_{kp}=\sum_{u=1}^{n}s_{uk}s_{up}=\sum_{u=1}^{n}0=0$$

若 $k=p=j$，由于 q_u 等概率地取 $\{1,2,...,m\}$ 中的一个值。因此 s_{uj} 是一个随机变量，有 $\frac{1}{m}$ 的概率等于 1，否则于 0，此时有：

$$\mathbb{E}[r_{kp}]=\sum_{u=1}^{n}\mathbb{E}[s_{uj}^2]=\sum_{u=1}^{n}\frac{1}{m}=\frac{n}{m}$$

综上，我们证明了 $\mathbb{E}[\boldsymbol{S}^T\boldsymbol{S}]=\frac{n}{m}\boldsymbol{I}_m$ 成立。

原命题成立，证毕。

2.4.3 SGD 算法的实现

参见./SGD/startcode.py 文件中的 stochastic_grad_descent 函数。

2.4.4 batch_size 调整

固定 $\lambda=0$，固定步长 $\eta=0.005$，调整 batch_size 的值，观察梯度下降算法的收敛情况。在训练集上的损失函数的变化曲线如图 3 所示，在验证集上的损失函数的变化曲线如图 4 所示。

我们测试了 batch_size 为 1、2、4、8、16、32、64 和 128 的情况。从图中可以看出，不同的 batch_size 下，算法的收敛速度几乎一致。但是，随着 batch_size 的增大，算法的收敛过程逐渐变得更平滑，波动幅度也逐渐减小。因此，我们认为 batch_size 越大，算法的收敛也就越稳定。

在 batch_size=128，初始步长为 0.01 时，我们取不同的步长调整策略，观察梯度下降算法的收敛情况。在训练集上的损失函数的变化曲线如图 5 所示，在验证集上的损失函数的变化曲线如图 6 所示。

可以看出，若取步长固定，算法的收敛速度较快，在训练集上的表现较为稳定，但在验证集上并不能收敛到最优参数位置，在后期甚至倾向于发

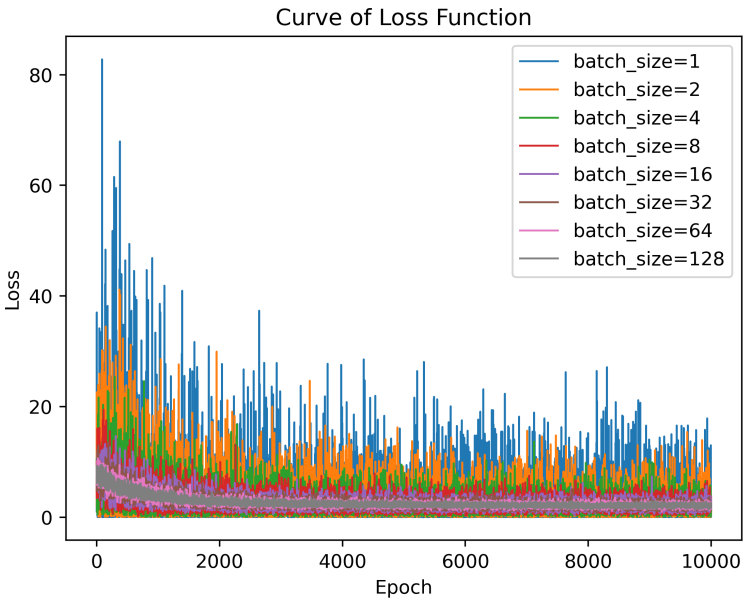

图 3: 不同批大小下训练集上的损失函数的变化曲线

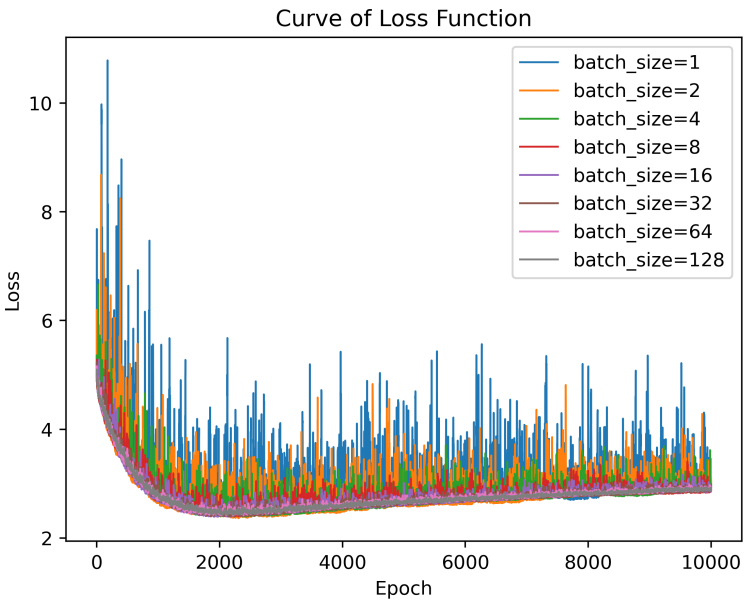

图 4: 不同批大小下验证集上的损失函数的变化曲线

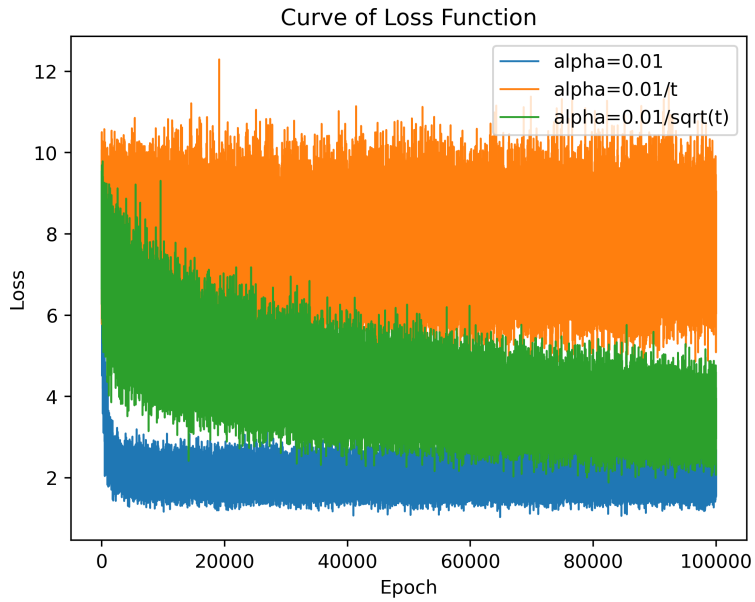

图 5: 不同步长调整策略下训练集上的损失函数的变化曲线

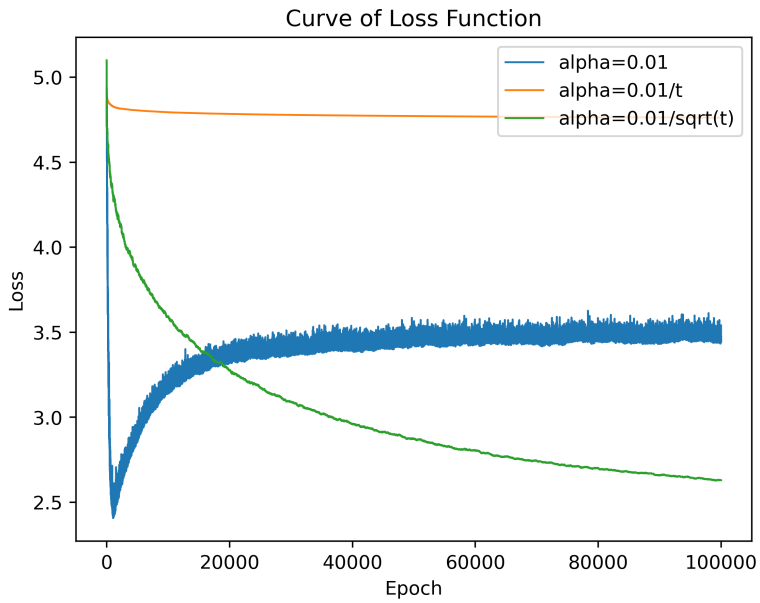

图 6: 不同步长调整策略下验证集上的损失函数的变化曲线

散；若取步长随迭代次数呈反比例变化，算法在验证集上的收敛过程变得非常光滑，但此时算法在后期的收敛速度过慢，导致不能够收敛到最优参数位置；而若取步长随迭代次数的平方根呈反比例变化，我们发现算法的性能良好，在训练集上能够以合适的速度和稳定性收敛，在验证集上也能够光滑、快速地收敛到最优的参数位置。

综上，取 batch_size=128，步长调整策略为随迭代次数的平方根呈反比例变化，SGD 算法的性能较好。

2.5 模型选择

使用 SGD 算法，取 batch_size=128，初始步长为 0.01，步长调整策略为随迭代次数的平方根呈反比例变化，迭代次数为 100000 次。

我们取正则化系数 $\lambda \in \{10^{-8}, 10^{-7}, ..., 10^{2}\}$，训练过程中模型在验证集上的损失函数值如图 8 所示，模型在验证集上的均方误差如表 1 所示。

表 1: 在不同的正则化系数下，模型在验证集上的均方误差

lambda	MSE (valid)
10^{-8}	2.629872330826207
10^{-7}	2.630012511937801
10^{-6}	2.629049094324194
$\mathbf{10^{-5}}$	**2.628083233220087**
10^{-4}	2.629858879741408
10^{-3}	2.631048055957441
10^{-2}	2.673634035806118
10^{-1}	3.124097256730932
10^{0}	4.429540373987043
10^{1}	4.909692882584656
10^{2}	5.061860288433802

从实验结果可以看出，当 $\lambda \leq 10^{-3}$ 时，模型误差较小。当 $\lambda = 10^{-5}$ 时，模型效果最好，**在验证集上的均方误差为 2.628**。

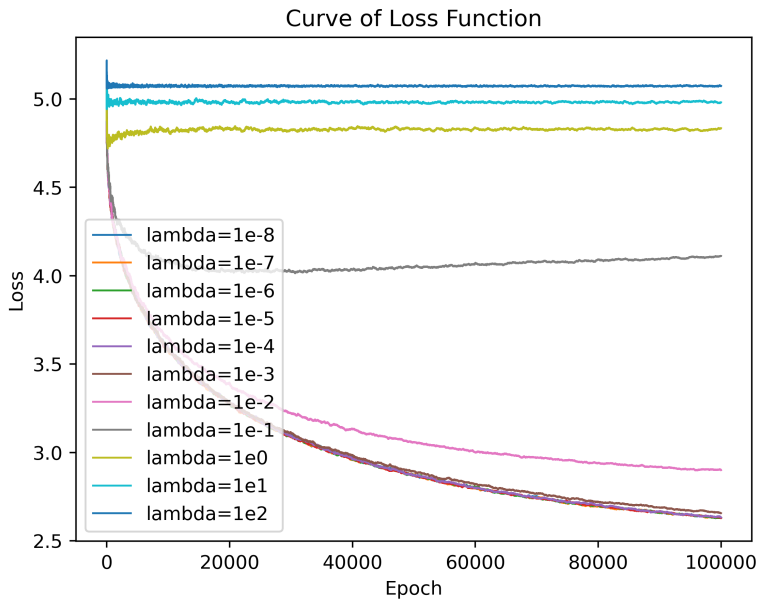

图 7: 不同正则化系数下验证集上的损失函数的变化曲线

2.6 牛顿法

2.6.1 目标函数的海森矩阵

根据题意，$J_{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\|\boldsymbol{X\theta} - \boldsymbol{y}\|_2^2 + \lambda\boldsymbol{\theta}^T\boldsymbol{\theta}$，有：

$$\nabla J_{MSE}(\boldsymbol{\theta}) = \frac{2}{m}\boldsymbol{X}^T(\boldsymbol{X\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta}$$

求微分：

$$d\nabla J_{MSE}(\boldsymbol{\theta}) = \frac{2}{m}\boldsymbol{X}^T\boldsymbol{X}d\boldsymbol{\theta} + 2\lambda d\boldsymbol{\theta} = (\frac{2}{m}\boldsymbol{X}^T\boldsymbol{X} + 2\lambda\boldsymbol{I})^T d\boldsymbol{\theta}$$

因此目标函数的海森矩阵为：

$$\boldsymbol{H} = \nabla^2 J_{MSE} = \frac{2}{m}\boldsymbol{X}^T\boldsymbol{X} + 2\lambda\boldsymbol{I} \tag{7}$$

任取非零向量 $\boldsymbol{x} \in \mathbb{R}^{d+1}$，由于 $\lambda > 0$，有：

$$\boldsymbol{x}^T\boldsymbol{H}\boldsymbol{x} = \boldsymbol{x}^T(\frac{2}{m}\boldsymbol{X}^T\boldsymbol{X} + 2\lambda\boldsymbol{I})\boldsymbol{x}$$

$$
\begin{aligned}
&= \frac{2}{m}(\boldsymbol{x}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{x}) + 2\lambda\boldsymbol{x}^T\boldsymbol{x} \\
&= \frac{2}{m}\|\boldsymbol{X}\boldsymbol{x}\|_2^2 + 2\lambda\|\boldsymbol{x}\|_2^2 \\
&\geq 2\lambda\|\boldsymbol{x}\|_2^2 > 0
\end{aligned}
$$

因此海森矩阵 $\boldsymbol{H}$ 在 $\lambda > 0$ 的情况下严格正定，因此一定非奇异。

2.6.2 基于牛顿法的梯度下降算法

略。

3 支持向量机

3.1 次梯度

3.1.1 合页损失函数的次梯度

定义合页损失函数：

$$
J(\boldsymbol{\omega}) = \max\{0, 1 - y\boldsymbol{\omega}^T\boldsymbol{x}\} \tag{8}
$$

合页损失函数的次梯度为：

$$
\partial J(\boldsymbol{\omega}) = \begin{cases} \boldsymbol{0} & \text{if} \quad y\boldsymbol{\omega}^T\boldsymbol{x} > 1 \\ -y\boldsymbol{x}^* & \text{if} \quad y\boldsymbol{\omega}^T\boldsymbol{x} = 1 \\ -y\boldsymbol{x} & \text{if} \quad y\boldsymbol{\omega}^T\boldsymbol{x} < 1 \end{cases} \tag{9}
$$

其中 $\boldsymbol{x}^*$ 与 $\boldsymbol{x}^*$ 形状一致，且满足：对 $\boldsymbol{x}^*$ 的任意一个分量，其值都在 0 与 $\boldsymbol{x}$ 的对应分量之间。

3.1.2 次梯度与凸性的关系

对于一个处处存在次梯度的函数 $f : \mathbb{R}^d \to \mathbb{R}$，反设 f 非凸，则存在 $\boldsymbol{x}_0$、$\boldsymbol{y}_0 \in \mathbb{R}^d$ 和 $\lambda_0 \in (0, 1)$，使得：

$$
\lambda_0 f(\boldsymbol{x}_0) + (1 - \lambda_0) f(\boldsymbol{y}_0) < f(\lambda_0\boldsymbol{x}_0 + (1 - \lambda_0)\boldsymbol{y}_0)
$$

考虑 $\lambda_0\boldsymbol{x}_0+(1-\lambda_0)\boldsymbol{y}_0$ 处的次梯度，有以下不等式立：

$$
\begin{aligned}
f(\boldsymbol{x}_0) &\geq f(\lambda_0\boldsymbol{x}_0+(1-\lambda_0)\boldsymbol{y}_0)+(1-\lambda_0)g^T(\boldsymbol{x}_0-\boldsymbol{y}_0)\\
f(\boldsymbol{y}_0) &\geq f(\lambda_0\boldsymbol{x}_0+(1-\lambda_0)\boldsymbol{y}_0)+\lambda_0 g^T(\boldsymbol{y}_0-\boldsymbol{x}_0)
\end{aligned}
$$

将两不等式分别乘以 λ_0 和 $1-\lambda_0$，相加得：

$$
\begin{aligned}
\lambda_0 f(\boldsymbol{x}_0)+(1-\lambda_0)f(\boldsymbol{y}_0) \geq\ & f(\lambda_0\boldsymbol{x}_0+(1-\lambda_0)\boldsymbol{y}_0)\\
&+\lambda_0(1-\lambda_0)g^T(\boldsymbol{x}_0-\boldsymbol{y}_0)\\
&+\lambda_0(1-\lambda_0)g^T(\boldsymbol{y}_0-\boldsymbol{x}_0)\\
=\ & f(\lambda_0\boldsymbol{x}_0+(1-\lambda_0)\boldsymbol{y}_0)
\end{aligned}
$$

这与我们的反证假设相矛盾，因此 f 是凸函数。

3.2 感知机

3.2.1 使用 SSGD 算法最小化感知机损失

考虑线性假设空间，预测值 $\hat{y}=\boldsymbol{\omega}^T\boldsymbol{x}$，定义感知损失为：

$$
L(\boldsymbol{\omega})=\max\{0,-y\boldsymbol{\omega}^T\boldsymbol{x}\}
$$

参考 (9)，取 $L(\boldsymbol{\omega})$ 的次梯度为：

$$
\partial L(\boldsymbol{\omega})=\begin{cases}\boldsymbol{0} & \text{if} \quad y\boldsymbol{\omega}^T\boldsymbol{x}>0\\ -y\boldsymbol{x} & \text{if} \quad y\boldsymbol{\omega}^T\boldsymbol{x}\leq 0\end{cases}
$$

在 SSGD 算法中，不妨令 batch_size 为 1，固定步长为 1。设随机选取的训练集样本序号为 i，权重更新公式可表示为：

$$
\begin{aligned}
\boldsymbol{\omega}(k+1) &= \boldsymbol{\omega}^{(k)}-\partial L(\boldsymbol{\omega})\\
&=\begin{cases}\boldsymbol{\omega}^{(k)} & \text{if} \quad y(\boldsymbol{\omega}^{(k)})^T\boldsymbol{x}_i>0\\ \boldsymbol{\omega}^{(k)}+y_i\boldsymbol{x}_i & \text{if} \quad y(\boldsymbol{\omega}^{(k)})^T\boldsymbol{x}_i\leq 0\end{cases}
\end{aligned}
$$

而在感知机算法中,权重的更新方式是:遍历数据集,若 $y(\boldsymbol{\omega}^{(k)})^T\boldsymbol{x}_i \leq 0$,则令 $\boldsymbol{\omega}^{(k+1)} = \boldsymbol{\omega}^{(k)} + y_i\boldsymbol{x}_i$，否则令 $\boldsymbol{\omega}^{(k+1)} = \boldsymbol{\omega}^{(k)}$。

同理于我们在 2.4.2 节中的证明，可以看出在期望意义下，两种算法的权重更新方式是完全一致的。

我们再证明两种算法的停机条件一致。

在 SSGD 算法中，损失函数 $L(\boldsymbol{\omega})$ 的最小值为 0，在 $\forall i \in \{1, 2, ...n\}$，均有 $y(\boldsymbol{\omega})^T\boldsymbol{x}_i \leq 0$ 时取到。这意味着：SSGD 算法的停机条件是 $y(\boldsymbol{\omega})^T\boldsymbol{x}_i \leq 0$，$i = 1, 2, ..., n$，与感知机算法中的停机条件一致。

综上，我们即证明了使用 SSGD 算法最小化感知机损失在一定条件下等价于感知机算法。

3.2.2 权重线性组合表示

由于 $\boldsymbol{\omega}^0 = \mathbf{0} = \sum_{i=1}^n 0 \cdot \boldsymbol{x}_i$，说明 $\boldsymbol{\omega}^0$ 是输入数据的线性组合。而在感知机算法中，$\boldsymbol{\omega}$ 的更新量只可能是 $y_j\boldsymbol{x}_j = (\sum_{i=1, i\neq j}^n 0 \cdot \boldsymbol{x}_i) + y_j \cdot \boldsymbol{x}_j$ 或 $\mathbf{0}$，也为输入数据的线性组合。因此，经过任意多次更新后，$\boldsymbol{\omega}$ 仍然是输入数据的线性组合。

3.3 软间隔支持向量机

软间隔支持向量机的 Primal Problem 为：

$$
\begin{aligned}
\min_{\boldsymbol{\omega}\in\mathbb{R}^d, b\in\mathbb{R}, \boldsymbol{\xi}\in\mathbb{R}^m} & \frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2 + \frac{1}{m}\sum_{i=1}^m \xi_i \\
s.t. \quad & y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i \\
& \xi_i \geq 0, i = 1, 2, ..., m
\end{aligned}
\tag{10}
$$

3.3.1 拉格朗日方程

软间隔支持向量机 Primal Problem 的拉格朗日方程为:

$$
\begin{aligned}
&L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \\
&\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2 + \frac{1}{m}\sum_{i=1}^{m}\xi_i + \sum_{i=1}^{m}\alpha_i(1-\xi_i - y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i + b)) - \sum_{i=1}^{m}\mu_i\xi_i \qquad (11)\\
&\alpha_i \geq 0, \mu_i \geq 0, i = 1, 2, ..., m
\end{aligned}
$$

3.3.2 对偶形式

Primal Problem 等价于求 $\min\limits_{\boldsymbol{\omega},b,\boldsymbol{\xi}} \max\limits_{\boldsymbol{\alpha},\boldsymbol{\mu}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})$，我们有:

$$
\min_{\boldsymbol{\omega},b,\boldsymbol{\xi}} \max_{\boldsymbol{\alpha},\boldsymbol{\mu}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) \geq \max_{\boldsymbol{\alpha},\boldsymbol{\mu}} \min_{\boldsymbol{\omega},b,\boldsymbol{\xi}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) \qquad (12)
$$

我们定义求解 $\max\limits_{\boldsymbol{\alpha},\boldsymbol{\mu}} \min\limits_{\boldsymbol{\omega},b,\boldsymbol{\xi}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu})$ 为软间隔支持向量机的 Dual Problem。由不等式 (12)，Dual Problem 的解是 Primal Problem 的一个近似解。

求解驻点条件:

$$
\begin{aligned}
\frac{\partial L}{\partial \boldsymbol{\omega}} &= \lambda\boldsymbol{\omega} - \sum_{i=1}^{m}\alpha_i y_i \boldsymbol{x}_i = \mathbf{0}\\
\frac{\partial L}{\partial b} &= \sum_{i=1}^{m}(-\alpha_i y_i) = 0\\
\frac{\partial L}{\partial \xi_i} &= \frac{1}{m} - \alpha_i - \mu_i = 0
\end{aligned}
$$

在 Dual Problem 的解中，应满足驻点条件，有:

$$
\begin{aligned}
\sum_{i=1}^{m}\alpha_i y_i \boldsymbol{x}_i &= \lambda\boldsymbol{\omega}\\
\sum_{i=1}^{m}\alpha_i y_i &= 0\\
\alpha_i + \mu_i &= \frac{1}{m}
\end{aligned}
$$

代入 Dual Problem 形式中：

$$
\begin{aligned}
\max_{\boldsymbol{\alpha},\boldsymbol{\mu}} \min_{\boldsymbol{\omega},b,\boldsymbol{\xi}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) &= \max_{\boldsymbol{\alpha},\boldsymbol{\mu}} \{\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2 + \sum_{i=1}^m \alpha_i - \sum_{i=1}^m \alpha_i y_i \boldsymbol{\omega}^T \boldsymbol{x}_i\} \\
&= \max_{\boldsymbol{\alpha}} \{\sum_{i=1}^m \alpha_i - \frac{1}{2\lambda}\|\sum_{i=1}^m \alpha_i y_i \boldsymbol{x}_i\|_2^2\} \\
s.t. \quad & \sum_{i=1}^m \alpha_i y_i = 0, 0 \le \alpha_i \le \frac{1}{m}, i = 1, 2, ..., m
\end{aligned}
$$

于是，软间隔支持向量机的对偶形式即为：

$$
\begin{aligned}
& \max_{\boldsymbol{\alpha}} \{\sum_{i=1}^m \alpha_i - \frac{1}{2\lambda}\|\sum_{i=1}^m \alpha_i y_i \boldsymbol{x}_i\|_2^2\} \\
s.t. \quad & \sum_{i=1}^m \alpha_i y_i = 0, 0 \le \alpha_i \le \frac{1}{m}, i = 1, 2, ..., m
\end{aligned}
$$

3.3.3 使用核函数的软间隔支持向量机

在引入非线性映射 Φ 和对应的核函数 $k(\cdot, \cdot)$ 后，软间隔支持向量机的 Primal Problem 变为：

$$
\begin{aligned}
\min_{\boldsymbol{\omega}\in\mathbb{R}^d, b\in\mathbb{R}, \boldsymbol{\xi}\in\mathbb{R}^m} & \frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2 + \frac{1}{m}\sum_{i=1}^m \xi_i \\
s.t. \quad & y_i(\boldsymbol{\omega}^T \Phi(\boldsymbol{x}_i) + b) \ge 1 - \xi_i \\
& \xi_i \ge 0, i = 1, 2, ..., m
\end{aligned} \tag{13}
$$

由 Representer Theorem，最优的权重 $\boldsymbol{\omega}$ 必然是样本输入的的线性组合，因此不妨设：$\boldsymbol{\omega} = \sum_{i=1}^m \gamma_i \Phi(\boldsymbol{x}_i)$，代入 Primal Problem 中可得：

$$
\begin{aligned}
\min_{\boldsymbol{\omega}\in\mathbb{R}^d, b\in\mathbb{R}, \boldsymbol{\xi}\in\mathbb{R}^m} & \frac{\lambda}{2}(\sum_{i=1}^m \sum_{j=1}^m \gamma_i \gamma_j k(\boldsymbol{x}_i, \boldsymbol{x}_j)) + \frac{1}{m}\sum_{i=1}^m \xi_i \\
s.t. \quad & y_i(\sum_{j=1}^m \gamma_j k(\boldsymbol{x}_i, \boldsymbol{x}_j)\Phi(\boldsymbol{x}_i) + b) \ge 1 - \xi_i \\
& \xi_i \ge 0, i = 1, 2, ..., m
\end{aligned} \tag{14}
$$

类似地，使用核函数后，Dual Problem 变为：

$$\begin{aligned} &\max_{\boldsymbol{\alpha}}\{\sum_{i=1}^{m}\alpha_i-\frac{1}{2\lambda}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j k(\boldsymbol{x}_i,\boldsymbol{x}_j)\} \\ s.t.\quad &\sum_{i=1}^{m}\alpha_i y_i=0, 0\le\alpha_i\le\frac{1}{m}, i=1,2,...,m \end{aligned} \tag{15}$$

3.3.4 使用次梯度下降算法求解 SVM

考虑 SVM 的损失函数为：

$$J_i(\boldsymbol{\omega},b)=\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2+\max\{0,1-y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\} \tag{16}$$

记 $l_i(\boldsymbol{\omega},b)=\max\{0,1-y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\}$，则 (16) 式又可表示为：

$$J_i(\boldsymbol{\omega},b)=\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2+l_i(\boldsymbol{\omega},b)$$

在 J_i 中，$\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2$ 在定义域上均连续可导，有：$\frac{\partial(\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2)}{\partial\boldsymbol{\omega}}=\lambda\boldsymbol{\omega}$，$\frac{\partial(\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2)}{\partial b}=\mathbf{0}$。

因此，我们只需验证 $\partial J_i\big|_{\boldsymbol{\omega}}=\begin{cases}-y_i\boldsymbol{x}_i & \text{if} \quad y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)<1\\ \mathbf{0} & \text{if} \quad y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\ge 1\end{cases}$ 是 l_i 关于 $\boldsymbol{\omega}$ 的次梯度，验证 $\partial J_i\big|_b=\begin{cases}-y_i & \text{if} \quad y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)<1\\ 0 & \text{if} \quad y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\ge 1\end{cases}$ 是 l_i 关于 b 的次梯度即可。

对于前者，考虑任意和 $\boldsymbol{\omega}$ 形状相同的 $\boldsymbol{\omega}_*$，记：

$$\Delta_{\boldsymbol{\omega}}(\boldsymbol{\omega}_*)=l_i(\boldsymbol{\omega}_*,b)-l_i(\boldsymbol{\omega},b)-(\partial J_i\big|_{\boldsymbol{\omega}}(\boldsymbol{\omega}))^T(\boldsymbol{\omega}_*-\boldsymbol{\omega})$$

我们下面证明 $\Delta_{\boldsymbol{\omega}}(\boldsymbol{\omega}_*)\ge 0$ 恒成立：

若 $y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\ge 1$，则：

$$\Delta_{\boldsymbol{\omega}}(\boldsymbol{\omega}_*)=l_i(\boldsymbol{\omega}_*,b)\ge 0$$

若 $y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)<1$，则：

$$\begin{aligned}\Delta_{\boldsymbol{\omega}}(\boldsymbol{\omega}_*) &= l_i(\boldsymbol{\omega}_*,b)-(1-y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b))+y_i\boldsymbol{x}_i^T(\boldsymbol{\omega}_*-\boldsymbol{\omega})\\ &= \max\{0,1-y_i(\boldsymbol{\omega}_*^T\boldsymbol{x}_i+b)\}-(1-y_i(\boldsymbol{\omega}_*^T\boldsymbol{x}_i+b))\\ &\geq 0\end{aligned}$$

对于后者，考虑任意 $b_*\in\mathbb{R}$，记：

$$\Delta_b(b_*)=l_i(\boldsymbol{\omega},b_*)-l_i(\boldsymbol{\omega},b)-\partial J_i\big|_b(b)(b_*-b)$$

我们下面证明 $\Delta_b(b_*)\geq 0$ 恒成立：

若 $y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\geq 1$，则：

$$\Delta_b(b_*)=l_i(\boldsymbol{\omega}_*,b)\geq 0$$

若 $y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)<1$，则：

$$\begin{aligned}\Delta_b(b_*) &= l_i(\boldsymbol{\omega},b_*)-(1-y_i(\boldsymbol{\omega}\boldsymbol{x}_i+b))+y_i(b_*-b)\\ &= \max\{0,1-y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b_*)\}-(1-y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b_*))\\ &\geq 0\end{aligned}$$

综上，我们即证明了题目中给出的 SVM 的损失函数次梯度的正确性。

3.3.5 使用 SSGD 算法训练 SVM 的伪代码

以下是使用 SSGD 算法训练 SVM 的伪代码，这里不妨考虑 batch_size 为 1 的情况。

3.3.6 带 L2 正则项的软间隔支持向量机

对于带 L2 正则项的软间隔支持向量机，其 Primal Problem 为：

$$\begin{aligned}\min_{\boldsymbol{\omega}\in\mathbb{R}^d,b\in\mathbb{R},\boldsymbol{\xi}\in\mathbb{R}^m}\ & \frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2+\frac{1}{m}\sum_{i=1}^m\xi_i^2\\ s.t.\quad & y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i+b)\geq 1-\xi_i, i=1,2,...,m\end{aligned}\tag{17}$$

Algorithm 1 Training SVM with SSGD

Input: $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), ..., (\boldsymbol{x}_m, y_m) \in \mathbb{R}^d \times \{1, -1\}$

Output: $\boldsymbol{\omega} \in \mathbb{R}^d, b \in \mathbb{R}$ which minimize loss function J

```
ω^(0) ← 0
b^(0) ← 0
for epoch = 1, 2, ..., T do                                   ▷ T 为迭代次数
    choose (x_i, y_i) from input randomly
    compute subgradient δ_ω ← ∂J_i|_ω(ω^(epoch−1)), δ_b ← ∂J_i|_b(b^(epoch−1))
    update ω^(epoch) ← (ω^(epoch−1) − ηδ_ω)                   ▷ η 为学习率
    update b^(epoch) ← (b^(epoch−1) − ηδ_b)
end for
return ω^(T), b^(T)
```

设 Primal Problem 中优化函数的最小值为 m_0，我们证明，一定存在 $\boldsymbol{\xi}$，满足 $\xi_i \geq 0$，$i = 1, 2, ..., m$，使得优化函数可以取到最小值 m_0。

任取一个使优化函数取到最小值 m_0 的解 $(\boldsymbol{\omega}^*, b^*, \boldsymbol{\xi}^*)$。我们取 $\boldsymbol{\xi}^{+*}$，满足 $\xi_i^{+*} = |\xi_i^*| \geq 0$，$i = 1, 2, ..., m$。显然有 $\frac{1}{m}\sum_{i=1}^m \xi_i^{*2} = \frac{1}{m}\sum_{i=1}^m \xi_i^{+*2}$ 成立。

而对于任意的 $i = 1, 2, ..., m$，注意到：$y_i(\boldsymbol{\omega}^{*T}\boldsymbol{x}_i^* + b^*) \geq 1 - \xi_i^* \geq 1 - \xi^{+*}$。

因此，$(\boldsymbol{\omega}^*, b^*, \boldsymbol{\xi}^{+*})$ 也是使优化函数取到最小值 m_0 的一组解，且 $\xi_i^{+*} \geq 0$，$i = 1, 2, ..., m$。

这也解释了为什么我们可以省略 $\xi_i \geq 0$ 这一约束。

3.3.7 带 L2 正则项的软间隔支持向量机的拉格朗日方程与对偶形式

带 L2 正则项的软间隔支持向量机 Primal Problem 的拉格朗日方程为：

$$
\begin{aligned}
&L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}) = \\
&\frac{\lambda}{2}\|\boldsymbol{\omega}\|_2^2 + \frac{1}{m}\sum_{i=1}^m \xi_i^2 + \sum_{i=1}^m \alpha_i(1 - \xi_i - y_i(\boldsymbol{\omega}^T\boldsymbol{x}_i + b)) \qquad (18) \\
&\alpha_i \geq 0, i = 1, 2, ..., m
\end{aligned}
$$

求解驻点条件：

$$
\begin{aligned}
\frac{\partial L}{\partial \boldsymbol{\omega}} &= \lambda \boldsymbol{\omega} - \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i = \mathbf{0} \\
\frac{\partial L}{\partial b} &= \sum_{i=1}^{m} (-\alpha_i y_i) = 0 \\
\frac{\partial L}{\partial \xi_i} &= \frac{2}{m} \xi_i - \alpha_i = 0
\end{aligned}
$$

在 Dual Problem 的解中，应满足驻点条件，有：

$$
\begin{aligned}
\sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i &= \lambda \boldsymbol{\omega} \\
\sum_{i=1}^{m} \alpha_i y_i &= 0 \\
\frac{m}{2} \alpha_i &= \xi_i
\end{aligned}
$$

代入 Dual Problem 形式中：

$$
\begin{aligned}
\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{\omega}, b, \boldsymbol{\xi}} L(\boldsymbol{\omega}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}) &= \max_{\boldsymbol{\alpha}} \{ \frac{\lambda}{2} \|\boldsymbol{\omega}\|_2^2 + \sum_{i=1}^{m} (-\frac{m}{4} \alpha_i^2 + \alpha_i) - \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{\omega}^T \boldsymbol{x}_i \} \\
&= \max_{\boldsymbol{\alpha}} \{ \sum_{i=1}^{m} (-\frac{m}{4} \alpha_i^2 + \alpha_i) - \frac{1}{2\lambda} \| \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i \|_2^2 \} \\
s.t. \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, ..., m
\end{aligned}
$$

于是，带 L2 正则项的软间隔支持向量机的对偶形式即为：

$$
\begin{aligned}
& \max_{\boldsymbol{\alpha}} \{ \sum_{i=1}^{m} (-\frac{m}{4} \alpha_i^2 + \alpha_i) - \frac{1}{2\lambda} \| \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i \|_2^2 \} \\
s.t. \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, ..., m
\end{aligned}
$$

3.4 情绪检测

3.4.1 使用 SSGD 算法训练线性 SVM 模型

已实现，参见./SVM/startcode.py 中的 linear_svm_subgrad_descent 函数。

3.4.2 超参数调整

调整超参数，我们希望训练得到的线性 SVM 模型能获得更好的性能。我们将通过四次单变量实验，分别对步长、正则化系数、批大小和迭代次数进行调整。在不同的超参数下，我们记录了模型在训练集和验证集上的准确率变化情况，如表 2、表 3、表 4、表 5 所示。

表 2: 调整步长，模型在训练集和验证集上的准确率变化情况

步长	正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.005	0.0001	1	60000	0.785890	0.749819
0.010	0.0001	1	60000	0.883436	0.827910
0.050	0.0001	1	60000	0.953988	**0.867679**
0.100	0.0001	1	60000	0.962883	0.841649

表 3: 调整正则化系数，模型在训练集和验证集上的准确率变化情况

步长	正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.050	0.00005	1	60000	0.957055	0.858279
0.050	**0.00010**	1	60000	0.953988	**0.867679**
0.050	0.00020	1	60000	0.950307	0.866233
0.050	0.00050	1	60000	0.942331	0.857556

综上，我们取步长为 0.05，正则化系数为 0.0001，批大小为 1，迭代次数为 60000 次时，模型的性能较好，**在训练集上的准确率为 95.40%，在验证集上的准确率为 86.77%**。

表 4: 调整批大小，模型在训练集和验证集上的准确率变化情况

步长	正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.050	0.00010	**1**	60000	0.953988	**0.867679**
0.050	0.00010	4	60000	0.957362	0.863341
0.050	0.00010	16	60000	0.958589	0.864787
0.050	0.00010	64	60000	0.956748	0.862617

表 5: 调整迭代次数，模型在训练集和验证集上的准确率变化情况

步长	正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.050	0.00010	1	10000	0.876994	0.814172
0.050	0.00010	1	30000	0.935890	0.859725
0.050	0.00010	1	**60000**	0.953988	**0.867679**
0.050	0.00010	1	100000	0.967791	0.866956

3.4.3 收敛更快速的优化算法

我们首先估测题述算法的收敛速度。

设优化目标函数为 $f(\boldsymbol{\omega})$，它是 λ-强凸的。由 λ-强凸性的定义，有以下不等式成立：

$$f(\boldsymbol{\omega}^{(t)}) - f(\boldsymbol{\omega}^*) \leq \boldsymbol{\nu}_t^T(\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*) - \frac{\lambda}{2}\|\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*\|_2^2$$

根据题述算法的迭代过程，我们有 $\boldsymbol{\nu}_t = \lambda t(\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^{(t+1)})$，代入上式：

$$\begin{aligned}
&f(\boldsymbol{\omega}^{(t)}) - f(\boldsymbol{\omega}^*) \\
\leq&\lambda t(\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^{(t+1)})^T(\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*) - \frac{\lambda}{2}\|\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*\|_2^2 \\
=&\frac{\lambda t}{2}(\|\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*\|_2^2 - \|\boldsymbol{\omega}^{(t+1)} - \boldsymbol{\omega}^*\|_2^2 + \|\boldsymbol{\omega}^{(t+1)} - \boldsymbol{\omega}^{(t)}\|_2^2) - \frac{\lambda}{2}\|\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*\|_2^2 \\
\leq&\frac{\lambda}{2}((t-1)\|\boldsymbol{\omega}^{(t)} - \boldsymbol{\omega}^*\|_2^2 - t\|\boldsymbol{\omega}^{(t+1)} - \boldsymbol{\omega}^*\|_2^2) + \frac{\lambda t}{2}\|\boldsymbol{\omega}^{(t+1)} - \boldsymbol{\omega}^{(t)}\|_2^2
\end{aligned}$$

对 t 求和：

$$\sum_{t=1}^{T}(f(\boldsymbol{\omega}^{(t)})-f(\boldsymbol{\omega}^*)) \leq -\frac{\lambda T}{2}\|\boldsymbol{\omega}^{(T+1)}-\boldsymbol{\omega}^*\|_2^2+\frac{\lambda}{2}\sum_{t=1}^{T}t\|\boldsymbol{\omega}^{(t+1)}-\boldsymbol{\omega}^{(t)}\|_2^2$$
$$\leq \frac{\lambda}{2}\sum_{t=1}^{T}t(\frac{\boldsymbol{\nu}_t}{\lambda t})^2=\frac{1}{2\lambda}\sum_{t=1}^{T}\frac{\boldsymbol{\nu}_t^2}{t}$$

在强凸函数中，次梯度 $\boldsymbol{\nu}$ 有界，因此存在常数 $C>0$，使得 $\boldsymbol{\nu}^2<C$ 恒成立。于是：

$$\sum_{t=1}^{T}(f(\boldsymbol{\omega}^{(t)})-f(\boldsymbol{\omega}^*)) \leq \frac{C}{2\lambda}\sum_{t=1}^{T}\frac{1}{t} \leq \frac{C}{2\lambda}(1+\log(T))$$

于是：

$$\frac{1}{T}\sum_{t=1}^{T}(f(\boldsymbol{\omega}^{(t)})-f(\boldsymbol{\omega}^*)) \leq \frac{C}{2\lambda}\frac{1+\log(T)}{T} \tag{19}$$

这即证明了题述算法的收敛速度是 $O(\frac{\log(T)}{T})$，小于一般的收敛速度 $O(\frac{1}{\sqrt{T}})$。

在./SVM/startcode.py 中的 linear_svm_subgrad_descent_lambda 函数有该算法的实现。除题述算法外，我们还实现了取权重平均为输出权重值的算法。

我们记录了两种算法在训练过程中的损失函数值的变化情况如图 8 所示。其中在普通算法中，我们考虑了步长为 0.05 和 0.001 两种情形。为了消去随机下降中的振荡现象，我们总是记录每 10 个 epoch 中最小一个的损失函数值。

可以看出，虽然强凸函数的优化算法在训练初期不太平稳，但是在约 300 次迭代后，其收敛情况已经趋于平稳，体现出超越一般算法的优化效率，最终还能使模型收敛到损失函数值更低的位置。而在强凸函数的优化算法的两种实现上，我们发现采用直接输出权重的方式效果更好。

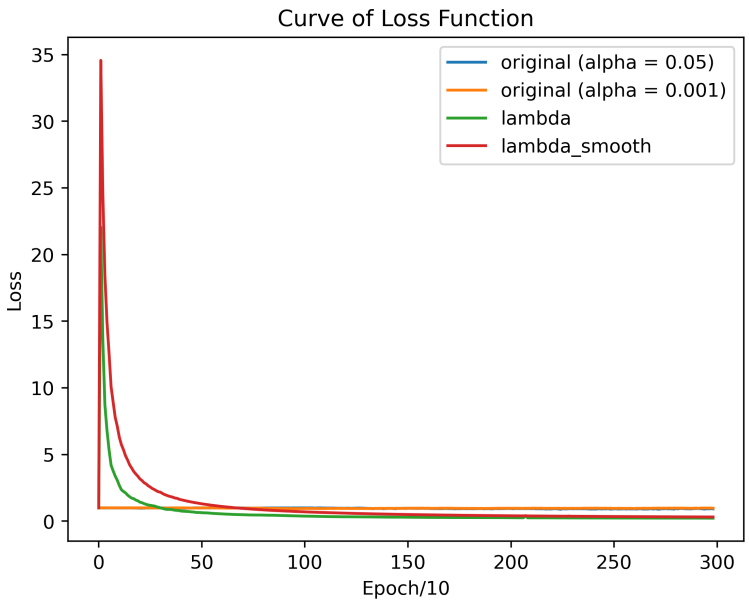

图 8: 不同算法下，SVM 模型在训练过程中的损失函数的变化曲线

3.4.4 基于核函数的 SVM

在./SVM/startcode.py 中的 kernel_svm_subgrad_descent 函数中，我们实现了基于径向核函数的 SVM 模型。

我们首先参考课件中的伪代码实现了第一个版本的算法。但是在调参过程中，我们发现算法难以收敛，推测问题出在步长策略上，基于径向核函数的 SVM 模型可能不能在固定步长下训练。

因此，我们参考了《Understanding Machine Learning: From Theory to Algorithms》一书中 16.3 节的有关内容，实现了第二个版本的算法，效果较好。该算法的伪代码如图 9 所示。

表 6、表 7、表 8 是调参的实验结果，我们主要调整了正则化系数、迭代次数和批大小三个超参数。

综上，我们取正则化系数为 0.001，批大小为 64，迭代次数为 5000 次时，模型的性能较好，**在训练集上的准确率为 99.54%，在验证集上的准确**

```
SGD for Solving Soft-SVM with Kernels
```

Goal: Solve Equation (16.5)
parameter: T
Initialize: $\boldsymbol{\beta}^{(1)} = \mathbf{0}$
for $t = 1, \ldots, T$
  Let $\boldsymbol{\alpha}^{(t)} = \frac{1}{\lambda t}\boldsymbol{\beta}^{(t)}$
  Choose i uniformly at random from $[m]$
  For all $j \neq i$ set $\beta_j^{(t+1)} = \beta_j^{(t)}$
  If $(y_i \sum_{j=1}^{m} \alpha_j^{(t)} K(\mathbf{x}_j, \mathbf{x}_i) < 1)$
    Set $\beta_i^{(t+1)} = \beta_i^{(t)} + y_i$
  Else
    Set $\beta_i^{(t+1)} = \beta_i^{(t)}$
Output: $\bar{\mathbf{w}} = \sum_{j=1}^{m} \bar{\alpha}_j \psi(\mathbf{x}_j)$ where $\bar{\boldsymbol{\alpha}} = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{\alpha}^{(t)}$

图 9: 改进的基于径向核函数的 SVM 模型的优化算法

表 6: 调整批大小，模型在训练集和验证集上的准确率变化情况

正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.001	1	1000	0.818405	0.746204
0.001	4	1000	0.882209	0.809111
0.001	16	1000	0.920552	0.836587
0.001	**64**	1000	0.939877	**0.852495**

表 7: 调整正则化系数，模型在训练集和验证集上的准确率变化情况

正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.0001	64	1000	0.945092	0.845987
0.0005	64	1000	0.938037	0.840202
0.0010	64	1000	0.939877	**0.852495**
0.0050	64	1000	0.927607	0.845987

表 8: 调整迭代次数，模型在训练集和验证集上的准确率变化情况

正则化系数	批大小	迭代次数	Acc (train)	Acc (valid)
0.001	64	500	0.899387	0.825741
0.001	64	1000	0.939877	0.852495
0.001	64	2000	0.976074	0.854664
0.001	64	**5000**	0.995399	**0.869848**

率为 86.98%。

我们发现，相比于一般的 SVM 模型，在引入核函数后，模型的在训练集上的准确率明显提升（95.40% -> 99.54%），说明核函数提高了模型的拟合能力。在验证集上，引入核函数后模型的准确率有一定的提升（86.77% -> 86.98%），但是提升幅度并不明显。我们认为可能的原因有两点：

1. 我们输入的词向量空间维度很高，导致词向量在空间中的分布非常稀疏。因此，线性超平面的分类效果和采用基函数变换后的超平面分类效果差距并不明显。

2. 注意到在引入核函数后，模型在训练集上的准确率已经接近 100%，这说明模型此时已经几乎将训练集中的数据完全分类。在不引入更多的先验知识的情况下，SVM 模型的性能将很难再进一步提高了。

3.4.5 最终模型的性能总结

使用基于径向核函数的 SVM 模型，取正则化系数为 0.001，批大小为 64，迭代次数为 10000 次。我们以此作为我们的最终模型。

在验证集上，其混淆矩阵为 $\begin{bmatrix} 609 & 92 \\ 98 & 584 \end{bmatrix}$**；准确率为 86.51%，精确率为 86.88%，召回率为 86.14%；F1-Score 等于 0.8651。**